Python OOP Interview-Style Problem Statement

Online Grocery Order Management System

Problem Context

A company is building a small online grocery ordering system like BigBasket, Blinkit, or Zepto. Customers can order grocery items from the app. The system should store customer details, store grocery item details, check item stock before order, check customer wallet balance before payment, apply discount based on customer type, reduce stock only after successful payment, reduce wallet balance only after successful payment, and show proper order status.

Concepts To Use

- Class
- Object
- Methods
- Multiple class linking
- Encapsulation
- Private variables
- Inheritance
- Method overriding
- Business validation

Business Scenario

There are two types of customers:

- NormalCustomer
- PremiumCustomer

Both customer types have common details:

- name
- phone
- city
- wallet balance

Discount rules:

- NormalCustomer - 0% discount
- PremiumCustomer - 10% discount

Premium customers also have one extra detail:

- membership_id

The grocery store has items like Rice, Milk, Oil, Sugar, and Wheat. Each grocery item has item name, category, price, and stock quantity. A customer can place an order for one grocery item.

Classes To Create

- Customer
- PremiumCustomer

- GroceryItem
- Order

1. Customer Class

Create a class called Customer.

Customer should have:

- name
- phone
- city
- wallet_balance

Important Rule

Wallet balance should be private because it is sensitive data. Do not store it as self.wallet_balance. Store it as self.__wallet_balance.

Customer should have these methods:

- display_customer_details() - prints customer name, phone, city, and wallet balance.
- get_wallet_balance() - returns the current wallet balance.
- pay_from_wallet(amount) - tries to make payment from the customer wallet and returns True or False.
- get_discount_percentage() - returns 0 for normal customer.

Rules for pay_from_wallet(amount):

- If amount is less than or equal to 0, payment should fail.
- If amount is less than or equal to wallet balance, payment should succeed and wallet balance should reduce.
- If amount is greater than wallet balance, payment should fail.
- Return True if payment succeeds.
- Return False if payment fails.

2. PremiumCustomer Class

Create a class called PremiumCustomer. This class should inherit from Customer. PremiumCustomer should reuse the common customer details from the Customer class.

PremiumCustomer should have one extra detail:

- membership_id

PremiumCustomer should override this method:

- get_discount_percentage() - returns 10 because premium customers get 10% discount.

PremiumCustomer should also have display_customer_details() showing customer name, phone, city, wallet balance, and membership ID.

3. GroceryItem Class

Create a class called GroceryItem.

GroceryItem should have:

- item_name
- category

- price
- stock_quantity

Important Rule

Price and stock quantity should be private because they are sensitive business values. Use self.__price and self.__stock_quantity.

GroceryItem should have these methods:

- display_item_details() - prints item name, category, price, and available stock.
- get_price() - returns item price.
- get_stock_quantity() - returns available stock quantity.
- reduce_stock(quantity) - reduces stock after successful order and returns True or False.

Rules for reduce_stock(quantity):

- If quantity is less than or equal to 0, stock should not reduce and method should return False.
- If quantity is less than or equal to available stock, stock should reduce and method should return True.
- If quantity is greater than available stock, stock should not reduce and method should return False.

4. Order Class

Create a class called Order. The Order class should connect Customer and GroceryItem. An order means which customer ordered which grocery item and how much quantity.

Order should have:

- customer
- grocery_item
- quantity
- total_amount
- discount_amount
- final_amount
- order_status

Initial order status should be Pending.

Order should have this method:

- place_order()
- display_order_details()

place_order() Flow

Step 1: Check quantity

If quantity is less than or equal to 0, order should fail. Order status should become Failed - Invalid Quantity. Payment should not happen. Stock should not reduce.

Step 2: Check stock

If requested quantity is greater than available stock, order should fail. Order status should become Failed - Stock Not Available. Payment should not happen. Stock should not reduce.

Step 3: Calculate total amount

Formula: total_amount = item price * quantity

Example: Rice price = 60, Quantity = 5, Total amount = 60 * 5 = 300

Step 4: Apply discount

Discount should depend on customer type. NormalCustomer gets 0% discount. PremiumCustomer gets 10% discount.

Formula: discount_amount = total_amount * discount_percentage / 100

Example: Total amount = 1000, Discount = 10%, Discount amount = 1000 * 10 / 100 = 100

Step 5: Calculate final amount

Formula: final_amount = total_amount - discount_amount

Example: Total amount = 1000, Discount amount = 100, Final amount = 900

Step 6: Make payment

Use the customer wallet payment method. If wallet balance is enough, payment should succeed and wallet balance should reduce. If wallet balance is not enough, order should fail. Order status should become Failed - Payment Failed. Stock should not reduce.

Step 7: Reduce stock

Stock should reduce only if payment is successful. If payment fails, stock should not reduce.

Step 8: Update order status

If everything is successful, order status should become Order Placed.

display_order_details()

This method should print customer name, item name, quantity, total amount, discount amount, final amount, and order status.

Test Cases Students Must Create

Test Case 1: Successful Normal Customer Order

Input:

- Customer Name: Ravi
- Wallet Balance: 1000
- Item: Rice
- Price: 60
- Stock: 20
- Quantity: 5

Expected:

- Total amount = 300
- Discount = 0
- Final amount = 300
- Payment successful
- Stock should reduce from 20 to 15
- Order status = Order Placed

Test Case 2: Successful Premium Customer Order

Input:

- Customer Name: Priya
- Wallet Balance: 1000
- Membership ID: PRM101
- Item: Oil
- Price: 200
- Stock: 10
- Quantity: 2

Expected:

- Total amount = 400
- Discount = 40
- Final amount = 360
- Payment successful
- Stock should reduce from 10 to 8
- Order status = Order Placed

Test Case 3: Failed Order Due To Low Wallet Balance

Input:

- Customer Name: Arjun
- Wallet Balance: 100

- Item: Sugar
- Price: 80
- Stock: 10
- Quantity: 3

Expected:

- Total amount = 240
- Wallet balance = 100
- Payment should fail
- Stock should not reduce
- Order status = Failed - Payment Failed

Test Case 4: Failed Order Due To Stock Not Available

Input:

- Customer Name: Meena
- Wallet Balance: 2000
- Item: Milk
- Price: 30
- Stock: 5
- Quantity: 10

Expected:

- Stock is not enough
- Payment should not happen
- Stock should remain 5
- Order status = Failed - Stock Not Available

Test Case 5: Failed Order Due To Invalid Quantity

Input:

- Customer Name: Kumar
- Wallet Balance: 1000
- Item: Wheat
- Price: 50
- Stock: 20
- Quantity: 0

Expected:

- Order should fail
- Payment should not happen
- Stock should not reduce
- Order status = Failed - Invalid Quantity

Important Rules

- Do not directly access wallet balance outside the Customer class.
- Do not directly access price outside the GroceryItem class.
- Do not directly access stock outside the GroceryItem class.
- Use methods to read or update private values.
- Stock should reduce only after successful payment.
- Wallet should reduce only after successful payment.
- Discount should come from customer type.
- PremiumCustomer should use method overriding.

Expected Submission

- Python code file or Google Colab notebook
- Output screenshot
- Short explanation of class design

Short Explanation Should Answer

- What classes did you create?
- Which variables are private?
- Why is wallet balance private?
- Why are price and stock private?
- How does Order connect Customer and GroceryItem?
- Where did you use inheritance?
- Where did you use method overriding?
- When does stock reduce?
- When does payment happen?

Bonus Challenge

After completing the basic problem, add one more customer type: SeniorCitizenCustomer.

- Senior citizen customers should get 5% discount.
- Extra detail: age
- If age is 60 or above, discount is 5%.
- If age is below 60, discount should be 0%.
- This should be done using inheritance and method overriding.

Final Reminder

This is a real business flow. Order should not be placed blindly. Stock should be checked. Payment should be checked. Discount should be applied based on customer type. Stock should reduce only after successful payment. Wallet should reduce only after successful payment.